Bitcoin Multi-Agent Advisor — Project Plan

Goal

A local tool that helps Ayham decide whether to take a Bitcoin trade. Ayham places trades manually; the system only advises. Built with Claude Code, also serving as a portfolio piece.

Agents

All agents use Claude Opus 5.5, run independently, and never talk to each other.

1. **Technical analysis** — price charts and indicators (RSI, moving averages, etc.).
2. **Derivatives** — funding rates, open interest, long/short ratios, liquidations (from OKX, Bybit, CoinGlass, or Binance public data).
3. **Macro** — US interest rates and Fed decisions, inflation, dollar index, Nasdaq/stocks, plus ETF flows. Data from FRED and Yahoo Finance; no web search needed.

Later options: news agent, public-analysts agent (pick analysts on older data to avoid selection bias), on-chain agent.

Scoring

Each agent produces two scores:

- **Rule-based score** — computed in code from explicit criteria (e.g. weak dollar adds X%, very high funding subtracts Y%).
- **Opus judgment score** — the model's own reasoning connecting the data, with a short explanation.

Raw numbers and indicators are always calculated in code, then handed to Opus.

Decision rule

No simple voting. Buy only when all three agents (or two, to be tested) are above a confidence threshold. Each agent's percentage is shown separately. Optionally test a debate version against the independent version.

Timeframe

Daily decisions.

RAG

Store past setups, decisions, and outcomes so agents can retrieve similar past situations. Fine-tuning is postponed until enough outcome data exists; outcomes act as free labels.

Backtesting

- Test period: July 2026 onward (after Opus 5.5's June 2026 training cutoff).
- Backtest each agent independently first; each should succeed on its own.
- Second check on older data with dates and coin name hidden.
- Calibration check: when an agent says 80%, is it right about 80% of the time?
- Compare against simply holding Bitcoin.

Metrics

Total profit, win rate, max drawdown.

Tracking

- API cost per agent, per run.
- Every prompt version saved and linked to its results.
- Suggested stop loss shown with each signal.

Tech stack

- Python, Django, Django REST Framework for the API.
- Database: PostgreSQL (or SQLite to start).
- Frontend: Django templates, plain HTML/CSS, Chart.js for charts.
- Runs locally only, no login.

Website pages

1. Daily dashboard — today's three scores and explanations.
2. Agent details — each agent's data, both scores, reasoning.
3. Backtest results — metrics per agent, calibration, vs holding.
4. Costs — API spend per agent.
5. Prompt history — versions and their results.

Optional extras

- Daily Telegram message with the scores.
- Log Ayham's own decisions next to the agents' to compare.

Rough cost estimate (unverified)

Three agents, daily runs: a few dollars to roughly $10–30 per month in Opus API calls, depending on input size.

Scoring rules per agent (starting draft)

Each agent's rule-based score starts at 50% and criteria add or subtract. Weights below are starting guesses to be tuned by backtesting.

Technical analysis

- Price above 50-day and 200-day moving average: +10% each; below: -10% each.
- RSI under 30 (oversold): +10%; over 70 (overbought): -10%.
- MACD bullish cross in last 3 days: +5%; bearish cross: -5%.
- Price near a support level: +5%; near resistance: -5%.

Derivatives

- Funding rate very high (crowded longs): -15%; negative (crowded shorts): +10%.
- Open interest rising with price: +5%; rising while price falls: -10%.
- Large long liquidations just happened (flush done): +10%.
- Long/short ratio extreme on either side: score moves against the crowd by 10%.

Macro

- Dollar index (DXY) falling over 2 weeks: +10%; rising: -10%.
- Nasdaq above its 50-day average: +10%; below: -10%.
- Fed rate cut expected or just happened: +10%; hike: -15%.
- Inflation (CPI) below expectations: +5%; above: -5%.
- Spot Bitcoin ETF net inflows over 5 days positive: +10%; negative: -10%.

Scores are capped between 0% and 100%.

Data sources

Agent	Data	Source
Technical	Daily BTC price (OHLCV)	Binance, OKX or Bybit public API, or CoinGecko
Derivatives	Funding, open interest, long/short ratio, liquidations	OKX / Bybit public API, CoinGlass
Macro	Fed rate, CPI, dollar index	FRED API (free key)
Macro	Nasdaq / S&P 500	Yahoo Finance (yfinance)
Macro	Spot BTC ETF flows	Farside Investors (daily table)

All data is pulled by Python code, cached in the database, and never fetched by the model itself.

Agent output format

Every agent returns JSON with these fields:

- `agent` : technical, derivatives or macro
- `date` : the decision date
- `rule_score` : 0–100, from code
- `opus_score` : 0–100, from Opus
- `direction` : bullish, bearish or neutral
- `reasons` : 3–5 short bullet reasons
- `key_data` : the numbers it used

- `suggested_stop_loss`: a price level
- `prompt_version`, `input_tokens`, `output_tokens`, `cost_usd`

Opus must reply only in this JSON; invalid output is retried once, then logged as an error.

Database tables

- **MarketData** — date, source, field name, value.
- **AgentRun** — date, agent, rule_score, opus_score, direction, reasons, key_data, stop loss, prompt version, tokens, cost.
- **Decision** — date, the three agents' scores, combined signal (buy / no trade), threshold used.
- **Outcome** — date, price at decision, price after 1, 3 and 7 days, correct or not per agent.
- **PromptVersion** — agent, version number, prompt text, created date, notes.
- **BacktestRun** — period, settings, win rate, total profit, max drawdown, vs buy-and-hold.
- **MemoryCase** (RAG) — past setup summary, embedding, outcome.
- **ManualTrade** (optional) — Ayham's own decision and result.

Build order (step by step)

Each step should work and be tested before the next one starts.

1. **Project setup** — Django project, one app per area (data, agents, backtest, dashboard), `.env` for API keys, SQLite to start.
2. **Database models** — create the tables above and run migrations.
3. **Data collectors** — one Python script per source; pull daily data from January 2025 to today and save it. Check for missing days.
4. **Indicator calculations** — code for moving averages, RSI, MACD, support/resistance, DXY trend, ETF flow sums. Unit-test each one.
5. **Rule-based scores** — code the scoring rules for all three agents. No AI yet.
6. **First backtest (rules only)** — run rule scores over July 2026 onward; compute win rate, profit, max drawdown vs holding. Free to run.
7. **Opus agents** — one prompt per agent, saved as PromptVersion 1; send the calculated data, get the JSON back; log tokens and cost.
8. **Opus backtest** — run each agent alone over July 2026 onward, day by day, giving it only data up to that day. Hide dates and the coin name.

9. **Calibration check** — group calls by confidence (50s, 60s, 70s, 80s) and check how often each group was right.
10. **Combining rule** — test thresholds (all three above X, or two above X) and pick the best on the backtest.
11. **REST API** — Django REST Framework endpoints for today's decision, agent runs, backtests, costs, prompts.
12. **Dashboard pages** — Django templates, HTML/CSS, Chart.js: daily dashboard, agent details, backtest results, costs, prompt history.
13. **Daily run** — a scheduled command that collects data, runs the agents, and saves the decision each morning.
14. **RAG memory** — store each day's setup and outcome; let agents retrieve the 3–5 most similar past cases.
15. **Optional** — Telegram message, manual trade log, debate version test, extra agents (news, analysts, on-chain).

Rules for Claude Code

- Never let a backtest see data from after the decision date.
- All math is done in code, never by the model.
- Every Opus call logs prompt version, tokens and cost.
- Changing a prompt creates a new PromptVersion, never edits the old one.
- No real trades are placed by the system.
- Ask before adding new libraries or paid services.

Code structure and readability

- No CrewAI or other agent framework: plain Python calls to the Anthropic API.
- Each agent has its own folder, e.g. `agents/technical/`, with `prompt.md` (the prompt, plain text), `rules.py` (rule-based score) and `agent.py` (the Opus call).
- Same layering as Chess Coach: View → Service → Repository/AI → Database; models split into separate files.
- A `CLAUDE.md` file at the root with requirements, conventions, progress and current step.
- Short functions, clear names, a comment explaining why for anything non-obvious.

- Lessons from Chess Coach: no nested project folders; every app has `migrations/__init__.py` and `admin.py`; imports checked; `.env.example` matches settings exactly.
- After every step Claude Code runs `python manage.py check` and the tests, and explains each change before moving on.